

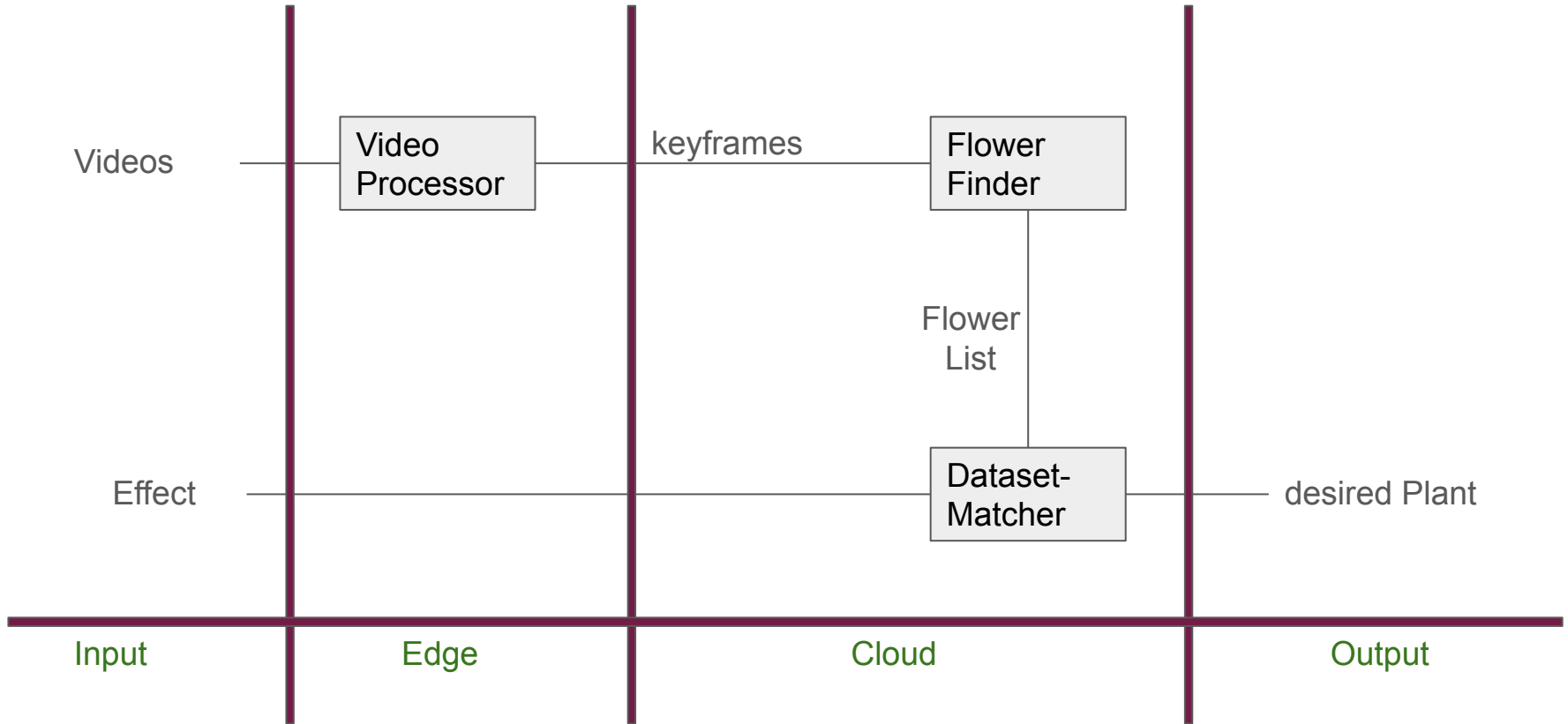
Flower Finder

Gabriele Tondi, Manuel Hoi and Yuliia Lomonosova

The Idea

- We desire some medicinal property
- There is a video of flowers
- The application tells us which flower in the video has the desired effect

Workflow



Dataset 1 - Flowers

- +1.000 classes detected with MobilNetV2 from Google

 astilbe 737 files	 bellflower 873 files	 black_eyed_susan 1000 files	 calendula 978 files	 california_poppy 1022 files	 carnation 923 files	 common_daisy 980 files	 coreopsis 1047 files
 daffodil 970 files	 dandelion 1052 files	 iris 1054 files	 magnolia 1048 files	 rose 999 files	 sunflower 1027 files	 tulip 1048 files	 water_lily 982 files



Carnation

**Common
Daisy**

Coreopsis

Dandelion

Iris

Sunflower

Dataset 2 - Medical Effects

Data from “Plants For A Future”

27 columns for ~18 000 entries

use_keyword
latin_name_search
common_name_search
edibility_rating_search
medicinal_rating_search
plant_url
Care Requirements
Common Name
Common Names
Cultivation Details

Edibility Rating
Edible Uses
Family
Image URLs
Known Hazards
Medicinal Properties
Medicinal Rating
Native Range
Other Uses
Other Uses Rating

Propagation
Range
Scientific Name
Special Uses
Summary
USDA hardiness
Weed Potential

Code 0 - The Repository

```
(flower-finder-env) PS C:\UNI\Distributed S
Elenco del percorso delle cartelle per il v
Numero di serie del volume: 2837-745C
C:..
```

```
| .gitattributes
| .gitignore
| docker-compose.yml
| environment.yml
| Flower finder.pdf
| install_docker.sh
| locustfile.py
| README.txt
| requirements.txt
| run_workflow.ps1
| setup_infra.ps1
| structure.txt
| sunflower.mp4
| training_model.py
```

```
+---dataset-matcher
|   analyze_dataset.py
|   database.py
|   Dockerfile
|   pfaf_plants_merged.csv
|   requirements.txt
```

```
+---flower-recognizer
|   Dockerfile
|   flower_classes.json
|   flower_classifier_model.h5
|   recognize_flower.py
|   requirements.txt
```

```
+---key_frame_output_folder
```

```
+---monitor results
```

```
|   \---first run - e2 medium - 10 users
```

```
|           VM Instance - CPU usage [RATE].png
```

```
|           VM Instance - CPU utilization [MEAN].png
```

```
|           VM Memory Used [MEAN].png
```

```
+---storage
```

```
|   +---input
```

```
|   +---processed
```

```
|   \---results
```

```
|           desired_effect.txt
```

```
|           recognized_flowers.json
```

```
+---video-processor
```

```
|   Dockerfile
```

```
|   README.md
```

```
|   requirements.txt
```

```
|   VideoProcessing.py
```

```
\---__pycache__
```

```
|   locustfile.cpython-310.pyc
```

```
|   locustfile.cpython-312.pyc
```

Code 1 - Scripts: setup_infra.ps1

```
1 # File: setup_infra.ps1 (FINAL and COMPLETE VERSION)
2 $ZONE = "europe-west1-b"
3 $SCOPES = "https://www.googleapis.com/auth/cloud-platform"
4 $REMOTE_USER = "Gabriele"
5
6 # Helper function for running commands on the VMs
7 function Invoke-GCloudSshCommand {
8     param (
9         [string]$Instance,
10         [string]$Command
11     )
12     gcloud compute ssh "${REMOTE_USER}@${Instance}" --zone=$ZONE --command=$Command
13     if ($LASTEXITCODE -ne 0) {
14         Write-Error "Command failed on ${Instance}. Halting script."
15         exit 1
16     }
17 }
18
19 Write-Host "--- Starting One-Time Infrastructure Setup ---"
20
21 # --- 1. Clean Up and Create Resources ---
22 Write-Host "[1/4] Cleaning up old resources and creating new VMs..."
23 gcloud compute instances delete edge-instance cloud-instance --zone=$ZONE --quiet
24 gcloud compute firewall-rules delete allow-ssh-iap --quiet
25 gcloud compute firewall-rules create allow-ssh-iap --direction=INGRESS --action=ALLOW --rules=tcp:22 --source-ranges=0.0.0.0/0
26 # ** Increased boot disk size for edge-instance **
27 gcloud compute instances create edge-instance --zone=$ZONE --machine-types="e2-small" --image-family="ubuntu-2204-lts" --image-projects="ubuntu-os-cloud" --scopes=$SCOPES --boot-disk-size=30GB
28 # ** Upgraded machine type and increased boot disk size for cloud-instance **
29 gcloud compute instances create cloud-instance --zone=$ZONE --machine-type="n2-standard-4" --image-family="ubuntu-2204-lts" --image-project="ubuntu-os-cloud" --scopes=$SCOPES --boot-disk-size=30GB
30
31 # --- 2. Manual SSH Handshake (User Action Required) ---
32 Write-Host "n--- ACTION REQUIRED ---"
33 Write-Host "The script will now pause. Please run the following two commands in a NEW terminal:"
34 Write-Host "1. gcloud compute ssh ${REMOTE_USER}@edge-instance --zone=${ZONE}"
35 Write-Host "2. gcloud compute ssh ${REMOTE_USER}@cloud-instance --zone=${ZONE}"
36 Write-Host "Answer 'y' to any prompts, then close the new windows and return here."
37 Read-Host -Prompt "Press Enter to continue AFTER you have completed the SSH steps..."
38
39 # --- 3. Install Dependencies and Set Permissions ---
40 Write-Host "[3/4] Installing dependencies and setting permissions on VMs..."
41 $INSTALL_COMMAND = "sudo apt-get update -y && sudo apt-get install -y git git-lfs && curl -fsSL https://get.docker.com -o get-docker.sh && sudo sh get-docker.sh"
42 Invoke-GCloudSshCommand -Instance "edge-instance" -Command $INSTALL_COMMAND
43 Invoke-GCloudSshCommand -Instance "cloud-instance" -Command $INSTALL_COMMAND
44 $PERMISSION_COMMAND = "sudo usermod -aG docker ${REMOTE_USER}"
45 Invoke-GCloudSshCommand -Instance "edge-instance" -Command $PERMISSION_COMMAND
46 Invoke-GCloudSshCommand -Instance "cloud-instance" -Command $PERMISSION_COMMAND
47
48 # --- 4. Pre-authorize SSH connection from Edge to Cloud ---
49 Write-Host "[4/4] Pre-authorizing SSH from Edge to Cloud..."
50 Invoke-GCloudSshCommand -Instance "edge-instance" -Command "ssh-keygen -t rsa -f ~/.ssh/id_rsa -q -N ""
51 $EdgePubKey = (gcloud compute ssh "${REMOTE_USER}@edge-instance" --zone=$ZONE --command="cat ~/.ssh/id_rsa.pub")
52 Invoke-GCloudSshCommand -Instance "cloud-instance" -Command "echo '$(EdgePubKey)' >> ~/.ssh/authorized_keys"
53
54 Write-Host "--- Infrastructure Setup Complete ---"
```

Code 1 - Scripts: run_workflow.ps1

```
param (
    [string]$VideoPath,
    [string]$Effect,
    [string]$GitRepoUrl,
    # Accept the UserID as a parameter
    [string]$UserID
)

$ZONE = "europe-west1-b"
$REMOTE_USER = "Gabriele"
# Create a unique project path for each user
$REMOTE_PROJECT_PATH = "/home/Gabriele/cloud-computing-project_$(UserID)"

function Invoke-GcloudSshCommand {
    param (
        [string]$Instance,
        [string]$Command
    )
    gcloud compute ssh "${REMOTE_USER}@${Instance}" --zone=$ZONE --command=$Command
    if ($LASTEXITCODE -ne 0) {
        Write-Error "Command failed on ${Instance}. Halting script."
        exit 1
    }
}

Write-Host "--- Starting Application Workflow ---"

# --- Step 1: Deploy and Upload ---
Write-Host "[1/4] Deploying code and inputs..."
Invoke-GcloudSshCommand -Instance "edge-instance" -Command "sudo rm -rf $REMOTE_PROJECT_PATH"
Invoke-GcloudSshCommand -Instance "cloud-instance" -Command "sudo rm -rf $REMOTE_PROJECT_PATH"
$DEPLOY_COMMAND = "GIT_LFS_SKIP_SMUDGE=1 git clone $GitRepoUrl $REMOTE_PROJECT_PATH && cd $REMOTE_PROJECT_PATH && mkdir -p storage/input storage/processed storage/results"
Invoke-GcloudSshCommand -Instance "edge-instance" -Command $DEPLOY_COMMAND
Invoke-GcloudSshCommand -Instance "cloud-instance" -Command $DEPLOY_COMMAND
gcloud compute scp $VideoPath "${REMOTE_USER}@edge-instance:$($REMOTE_PROJECT_PATH)/storage/input/" --zone=$ZONE
Invoke-GcloudSshCommand -Instance "cloud-instance" -Command "echo '$Effect' | tee $($REMOTE_PROJECT_PATH)/storage/results/desired_effect.txt"

# --- Step 2: Run Video Processor ---
Write-Host "[2/4] Processing video on Edge VM..."
$EDGE_DOCKER_COMMAND = "cd $($REMOTE_PROJECT_PATH); docker compose up --build video-processor"
Invoke-GcloudSshCommand -Instance "edge-instance" -Command $EDGE_DOCKER_COMMAND

# --- Step 3: Transfer Keyframes & Run Final Services ---
Write-Host "[3/4] Recognizing flowers and matching dataset on Cloud VM..."
# Get the internal IP of the cloud-instance
$CLOUD_IP = (gcloud compute instances describe cloud-instance --zone=$ZONE --format='get(networkInterfaces[0].networkIP)')
# Use a direct, pre-authorized scp command from edge to cloud
$SCP_COMMAND = "scp -r -o StrictHostKeyChecking=no ${REMOTE_PROJECT_PATH}/storage/processed/* ${REMOTE_USER}@${CLOUD_IP}:${REMOTE_PROJECT_PATH}/storage/processed/"
Invoke-GcloudSshCommand -Instance "edge-instance" -Command $SCP_COMMAND

# Run both services sequentially to get the final result
$CLOUD_DOCKER_COMMAND = "cd $($REMOTE_PROJECT_PATH); docker compose up --build flower-recognizer && docker compose up --build dataset-matcher"
Invoke-GcloudSshCommand -Instance "cloud-instance" -Command $CLOUD_DOCKER_COMMAND

# --- Step 4: Display the Final Result ---
Write-Host "`n--- Final Result ---"
$LOGS_COMMAND = "cd $($REMOTE_PROJECT_PATH); docker compose logs --no-log-prefix --tail='20' dataset-matcher"
Invoke-GcloudSshCommand -Instance "cloud-instance" -Command $LOGS_COMMAND
Write-Host "-----"
Write-Host "--- Workflow Complete ---"
Invoke-GcloudSshCommand -Instance "edge-instance" -Command "sudo rm -rf $REMOTE_PROJECT_PATH"
Invoke-GcloudSshCommand -Instance "cloud-instance" -Command "sudo rm -rf $REMOTE_PROJECT_PATH"
```


Code 2 - Video Processor

```
10 def extract_keyframes(video_path, output_folder, threshold=25, max_time_interval=5.0):
11     # At the start of extract_keyframes
12     print(f"[METRIC] START video_processing for {video_path}")
13     """Extracts keyframes from a single video file."""
14     cap = cv2.VideoCapture(video_path)
15     if not cap.isOpened():
16         print(f"Error: Unable to open video: {video_path}")
17         return
18
19     fps = cap.get(cv2.CAP_PROP_FPS)
20     if fps == 0: fps = 25 # Default FPS if unable to determine
21
22     max_frames_interval = int(max_time_interval * fps)
23
24     if not os.path.exists(output_folder):
25         os.makedirs(output_folder)
26
27     ret, prev_frame = cap.read()
28     if not ret:
29         print(f"Error: Could not read the first frame of {video_path}")
30         cap.release()
31         return
32
33     prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)
34     cv2.imwrite(os.path.join(output_folder, "keyframe_0000.jpg"), prev_frame)
```

```
36     count = 1
37     frame_num = 0
38     last_keyframe_num = 0
39
40     while True:
41         ret, frame = cap.read()
42         if not ret:
43             break
44
45         frame_num += 1
46         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
47         diff = cv2.absdiff(gray, prev_gray)
48
49         if (np.mean(diff) > threshold) or (frame_num - last_keyframe_num >= max_frames_interval):
50             cv2.imwrite(os.path.join(output_folder, f"keyframe_{count:04d}.jpg"), frame)
51             count += 1
52             last_keyframe_num = frame_num
53
54         prev_gray = gray
55
56     cap.release()
57     print(f"Extracted {count} keyframes from {os.path.basename(video_path)}")
58     # At the end of extract_keyframes
59     print(f"[METRIC] END video_processing for {video_path}")
60
```

Code 2 - Flower Finder

```
51 if __name__ == '__main__':
52     if len(sys.argv) < 2:
53         print("Usage: python recognize_flower.py <path_to_image_directory>")
54         sys.exit(1)
55
56     input_directory = sys.argv[1]
57     output_dir = os.getenv('OUTPUT_DIR', 'storage/results')
58
59     if not os.path.exists(output_dir):
60         os.makedirs(output_dir)
61
62     final_flower_list = recognize_flowers_in_directory(input_directory)
63
64     output_filepath = os.path.join(output_dir, 'recognized_flowers.json')
65     with open(output_filepath, 'w') as f:
66         json.dump(final_flower_list, f, indent=2)
67
68     if final_flower_list:
69         print("\n--- Recognition Complete ---")
70         print("Unique flowers found:", final_flower_list)
71         print(f"Results saved to: {output_filepath}")
72     else:
73         print("\nNo flowers were recognized.")
```

Code 2 - Flower Finder(2)

```
37 def recognize_flowers_in_directory(directory_path):
38     """Processes all images in a directory and its subdirectories."""
39     recognized_flowers = set()
40     print(f"\nScanning images in directory: {directory_path}")
41     for root, dirs, files in os.walk(directory_path):
42         for filename in files:
43             if filename.lower().endswith(('.png', '.jpg', '.jpeg')):
44                 image_path = os.path.join(root, filename)
45                 flower_name = predict_flower(image_path)
46                 if flower_name:
47                     print(f"-> Found '{flower_name}' in {filename}")
48                     recognized_flowers.add(flower_name)
49     return list(recognized_flowers)
```

```
23 def predict_flower(image_path):
24     """Predicts the flower name from a single image path."""
25     try:
26         img = cv2.imread(image_path)
27         img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
28         im = cv2.resize(img_rgb, (IMAGE_SIZE, IMAGE_SIZE))
29         im = np.expand_dims(im, axis=0) / 255.0
30         prediction = model.predict(im, verbose=0)
31         predicted_class_index = np.argmax(prediction)
32         return class_names[predicted_class_index]
33     except Exception as e:
34         print(f"Could not process image {image_path}. Error: {e}")
35         return None
```

Code 3 - Dataset Matcher

```
51 def main():
52     """
53     Main function to run the matching process.
54     """
55     print("\n--- Task 3: Dataset Matcher ---")
56     print("Loading dataset...")
57     load_dataset("pfaf_plants_merged.csv")
58
59     # Get the input directory from an environment variable
60     input_dir = os.getenv('INPUT_DIR', 'storage/results')
61
62     recognized_flowers_path = os.path.join(input_dir, 'recognized_flowers.json')
63     desired_effect_path = os.path.join(input_dir, 'desired_effect.txt')
64
65     # 1. Load the list of recognized flowers from the previous step
66     try:
67         with open(recognized_flowers_path, 'r') as f:
68             recognized_flowers = json.load(f)
69             print(f"Successfully loaded recognized flowers: {recognized_flowers}")
70     except FileNotFoundError:
71         print(f"Error: Could not find '{recognized_flowers_path}'. Aborting.")
72         return
73
74     # 2. Load the user's desired effect from a text file
75     try:
76         with open(desired_effect_path, 'r') as f:
77             user_input = f.read().strip()
78             effects = parse_user_input(user_input)
79             print(f"Successfully loaded desired effects: {effects}")
80     except FileNotFoundError:
81         print(f"Error: Could not find '{desired_effect_path}'. Please create it. Aborting.")
82         return
83
84     if not recognized_flowers or not effects:
85         print("Input data is missing. Cannot perform matching.")
86         return
87
88     # 3. Find the first recognized flower that matches the desired effects
89     matching_flower = find_matching_flower(recognized_flowers, effects)
90
91     # 4. Print the final result
92     if matching_flower:
93         print("\n--- Match Found! ---")
94         print(f"The most suitable flower from your images is:")
95         print(f"  - Common Name: {matching_flower['common_name']}")
96         print(f"  - Scientific Name: {matching_flower['latin_name']}")
97         print("-----")
98     else:
99         print("\n--- No Match Found ---")
100        print("None of the flowers recognized in your images match the desired effects.")
101        print("-----")
102
103 if __name__ == "__main__":
104     main()
```

Code 3 - Dataset Matcher(2)

```
31 def find_matching_flower(recognized_flowers, required_effects):
32     """
33     Finds the first flower from the recognized list that satisfies all required effects.
34     """
35     required = set(effect.lower() for effect in required_effects)
36
37     # Iterate through the flowers that were actually found in the user's images
38     for flower_name in recognized_flowers:
39         flower_name_lower = flower_name.lower()
40         # Check if we have this flower in our benefits dataset
41         if flower_name_lower in plant_to_data:
42             latin_name, available_effects = plant_to_data[flower_name_lower]
43             # Check if this flower has all the effects the user wants
44             if required.issubset(available_effects):
45                 # Found a suitable flower!
46                 return {"common_name": flower_name, "latin_name": latin_name}
47
48     # If no match is found after checking all recognized flowers
49     return None
50
```


Docker compose

```
1  services:
2      video-processor:
3          build: ./video-processor
4          environment:
5              - INPUT_DIR=/app/storage/input
6              - OUTPUT_DIR=/app/storage/processed
7          volumes:
8              - ./storage/input:/app/storage/input
9              - ./storage/processed:/app/storage/processed
10
11     flower-recognizer:
12         build: ./flower-recognizer
13         environment:
14             - OUTPUT_DIR=/app/storage/results
15         volumes:
16             - ./storage/processed:/app/storage/processed:ro
17             - ./storage/results:/app/storage/results
18
19     dataset-matcher:
20         build: ./dataset-matcher
21         environment:
22             - INPUT_DIR=/app/storage/results
23         volumes:
24             - ./storage/results:/app/storage/results:ro
```

and Docker Files

```
1  # flower-recognizer/Dockerfile
2  FROM python:3.10-slim
3
4  # Install the system dependency for OpenCV
5  RUN apt-get update && apt-get install -y libgl1-mesa-glx libglib2.0-0
6
7  WORKDIR /app
8
9  COPY requirements.txt .
10 RUN pip install --no-cache-dir -r requirements.txt
11
12 COPY requirements.txt .
13 RUN pip install --no-cache-dir -r requirements.txt
14
15 COPY recognize_flower.py .
16 COPY flower_classes.json .
17
18 CMD ["python", "recognize_flower.py", "/app/storage/processed"]
```

and Docker Files

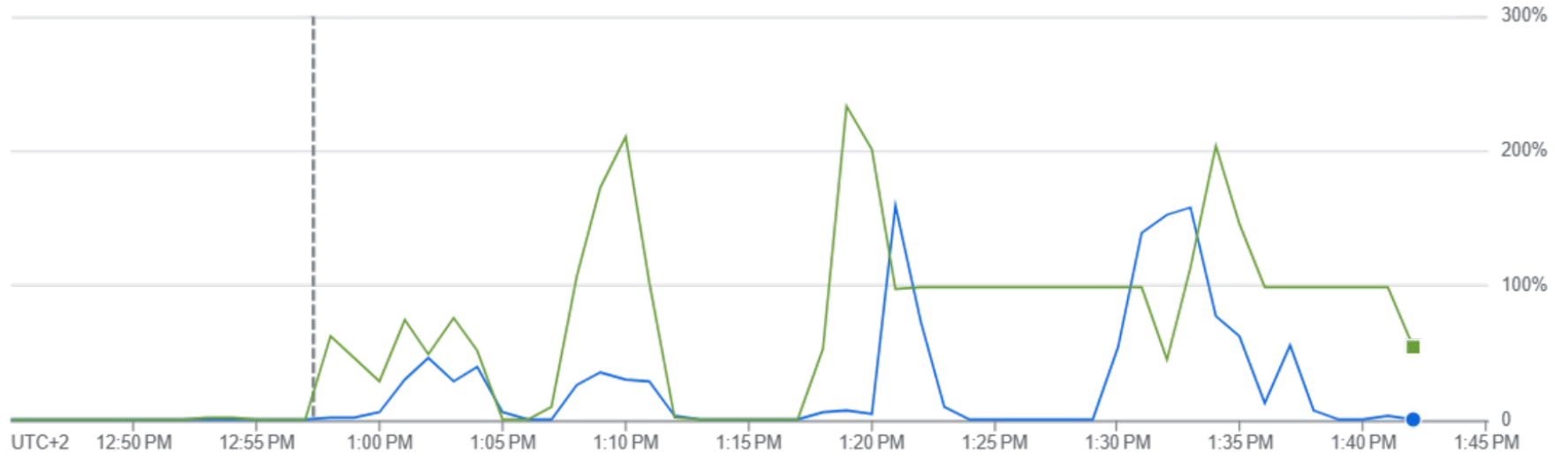
```
1  # dataset-matcher/Dockerfile
2  FROM python:3.10-slim
3
4  WORKDIR /app
5
6  COPY requirements.txt .
7  RUN pip install --no-cache-dir -r requirements.txt
8
9  COPY database.py .
10 COPY pfaf_plants_merged.csv .
11
12 CMD ["python", "database.py"]
```


and Docker Files

```
1  # video-processor/Dockerfile
2  FROM python:3.10-slim
3
4  # Install the system dependency for OpenCV
5  RUN apt-get update && apt-get install -y libgl1-mesa-glx libglib2.0-0
6
7  WORKDIR /app
8
9  COPY requirements.txt .
10 RUN pip install --no-cache-dir -r requirements.txt
11
12 COPY Video_Processing.py .
13
14 CMD ["python", "Video_Processing.py"]
```

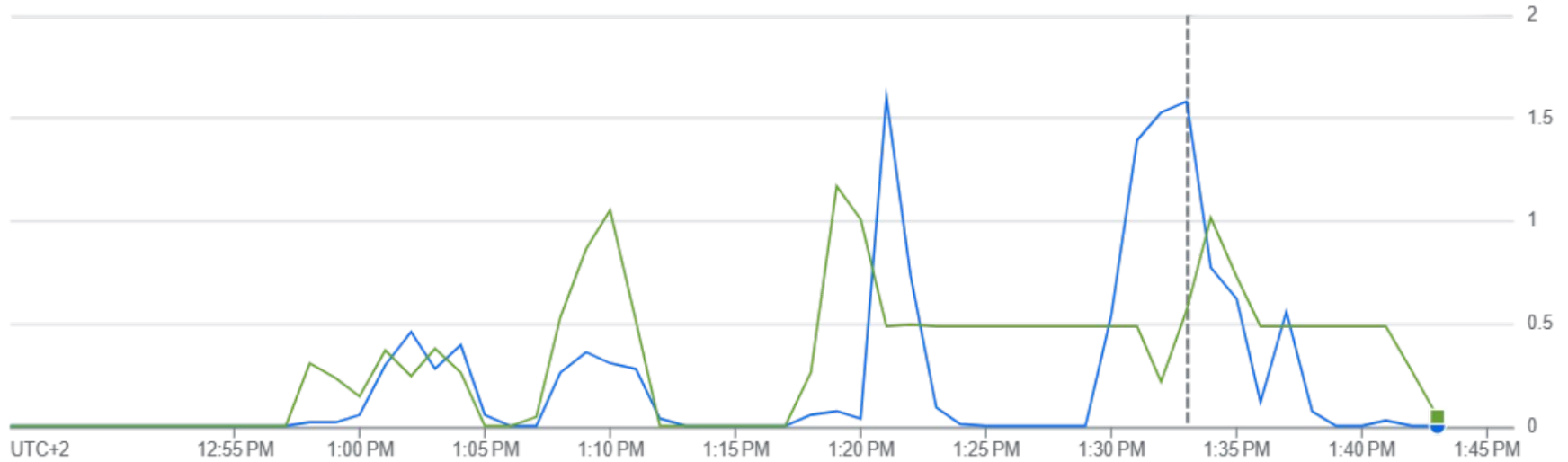
first run - e2 medium - 10 users

VM Instance - CPU utilization [MEAN]



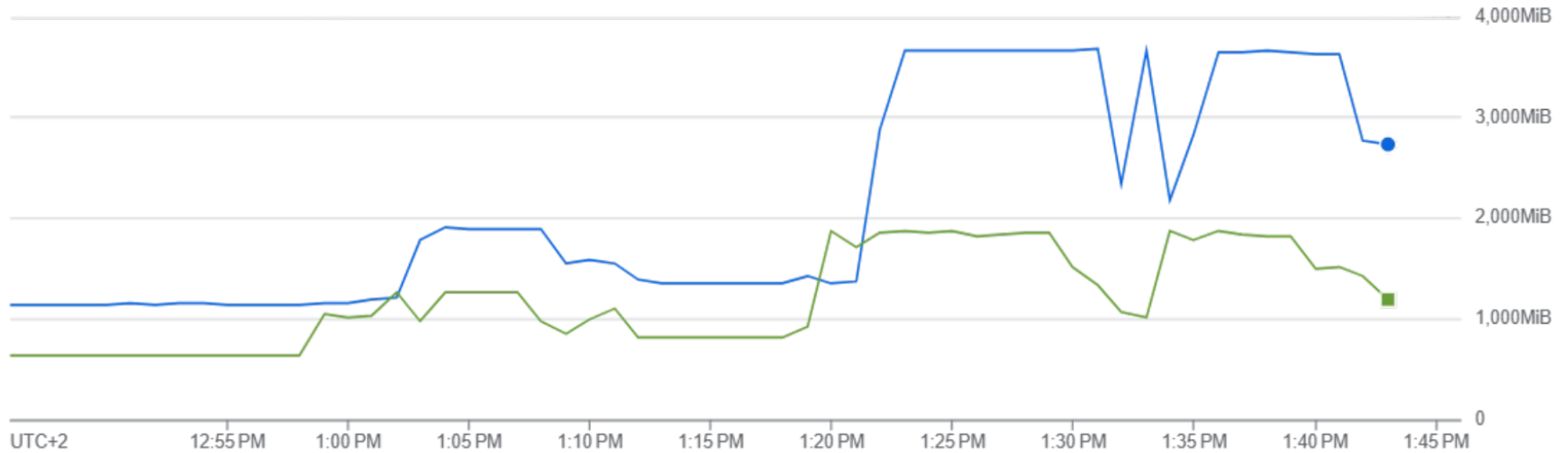
first run - e2 medium - 10 users

VM Instance - CPU usage [RATE]



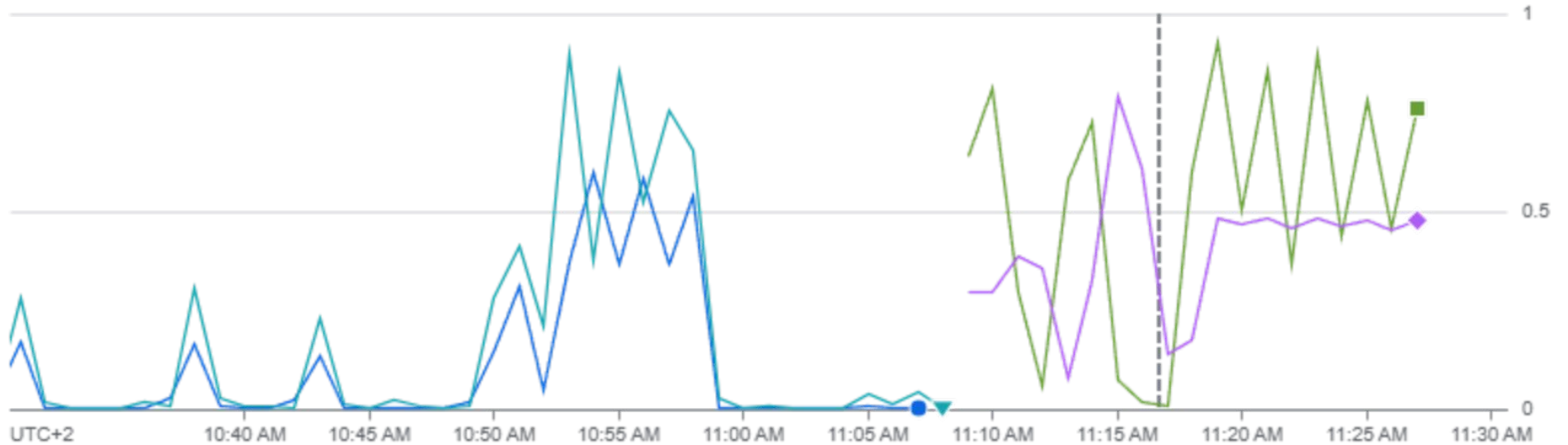
first run - e2 medium - 10 users

VM Memory Used [MEAN]



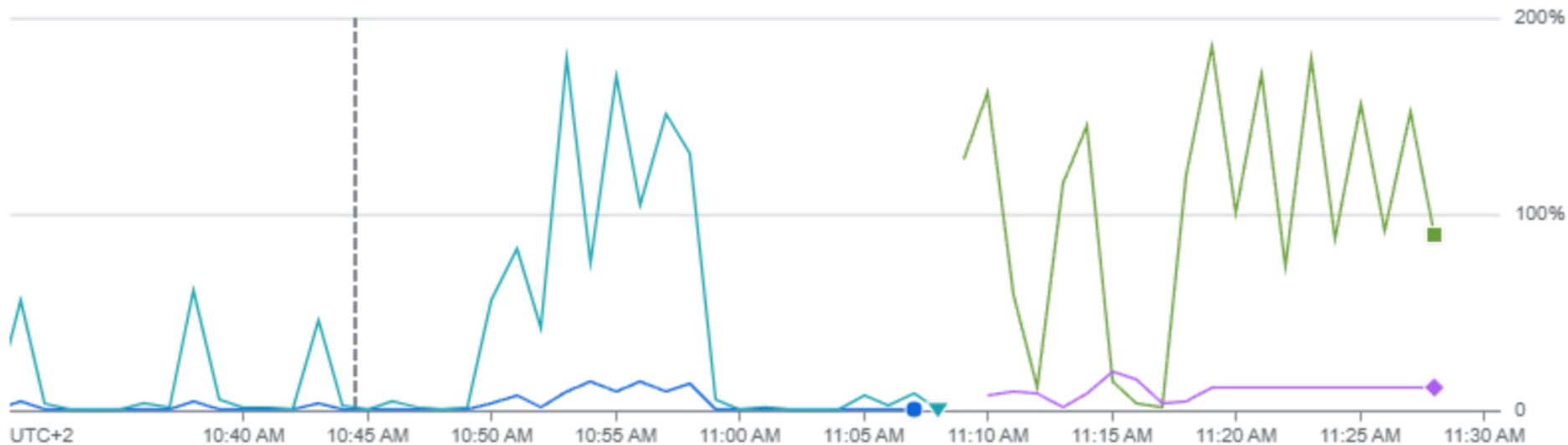
2 users - small edge - standard cloud

VM Instance - CPU usage [RATE]



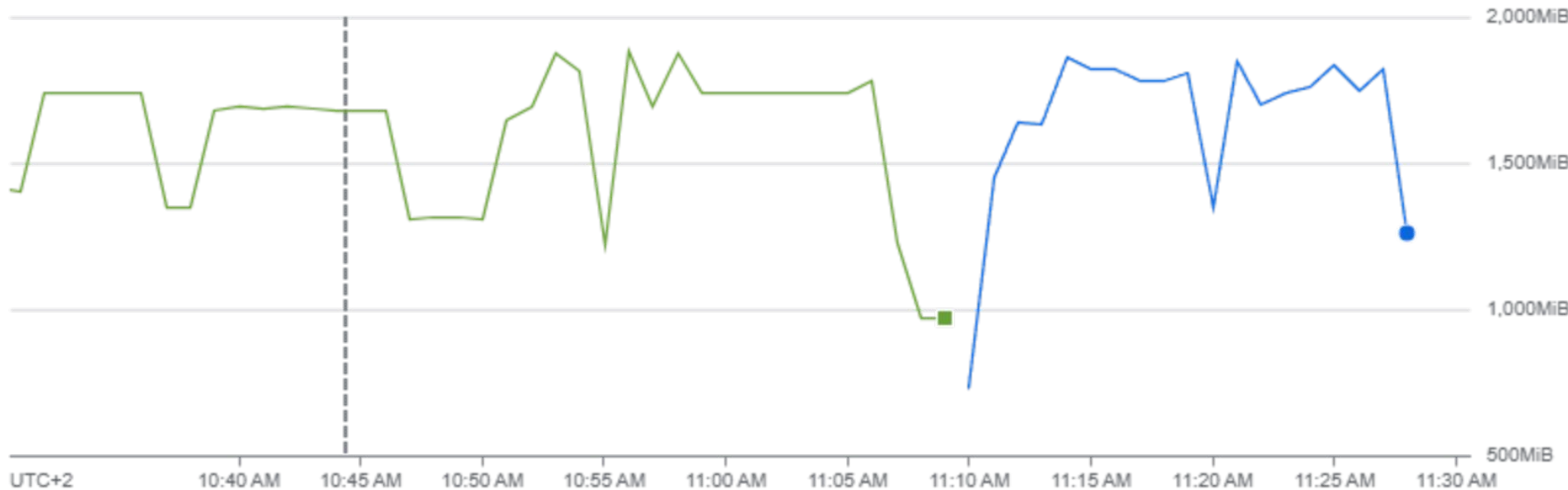
2 users - small edge - standard cloud

VM Instance - CPU utilization [MEAN]



2 users - small edge - standard cloud

VM Memory Used [MEAN]



2 users - small edge - standard cloud



Host

Status

running

Users

2

RPS

0.1

Failures

0%

Edit

Stop

Reset

Statistics

Charts

Failures

Exceptions

Current Ratio

Download Data

Logs

Locust Cloud

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
workflow	full_workflow	11	0	98000	126000	126000	102337.82	90196	125516	7633.09	0.1	0
	Aggregated	11	0	98000	126000	126000	102337.82	90196	125516	7633.09	0.1	0

About

10 users - small edge - standard cloud

VM Instance - CPU usage [RATE]



10 users - small edge - standard cloud

VM Instance - CPU utilization [MEAN]



10 users - small edge - standard cloud

VM Memory Used [MEAN]



10 users - small edge - standard cloud



Host

Status

running

Users

10

RPS

0.6

Failures

0%

Edit

Stop

Reset



Statistics

Charts

Failures

Exceptions

Current Ratio

Download Data

Logs

Locust Cloud

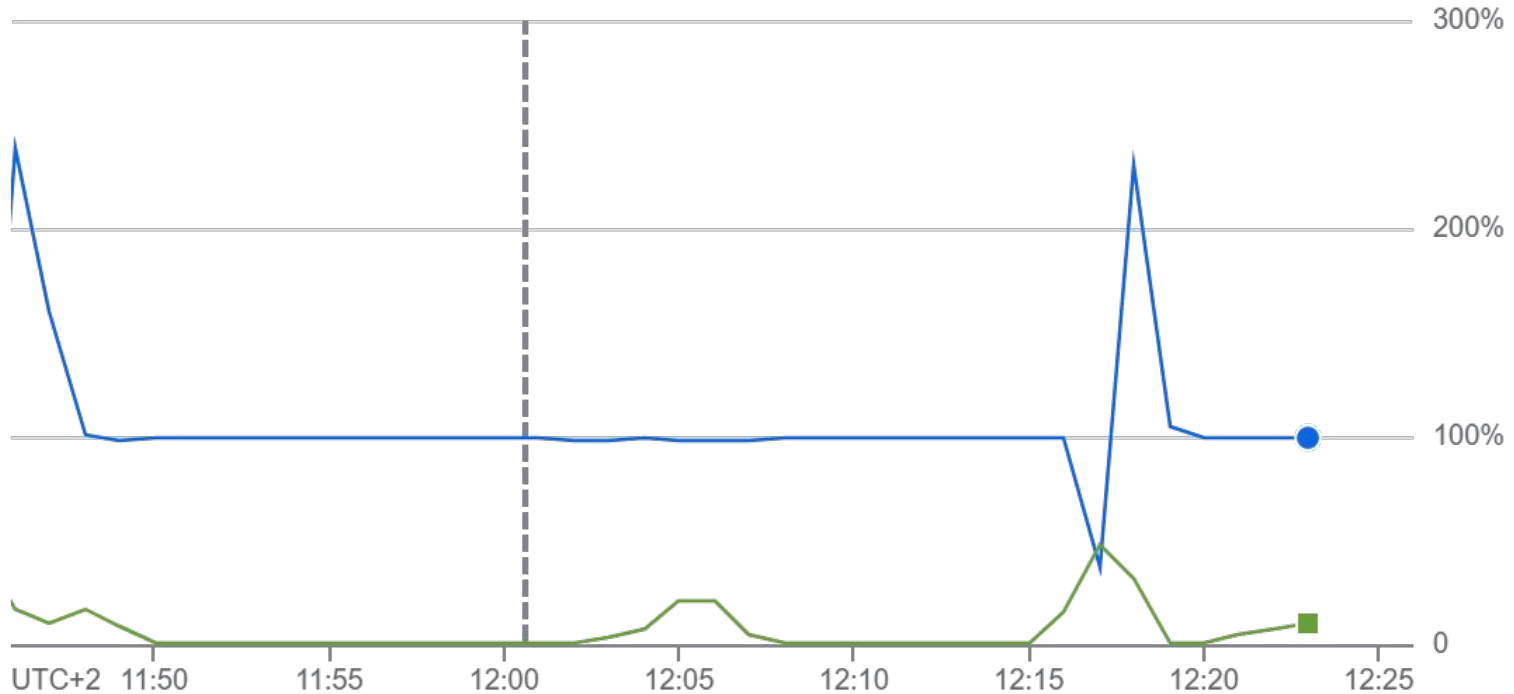


Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
workflow	full_workflow	9	0	743000	748000	748000	743341.11	739905	747555	11874.44	0.6	0
	Aggregated	9	0	743000	748000	748000	743341.11	739905	747555	11874.44	0.6	0

About

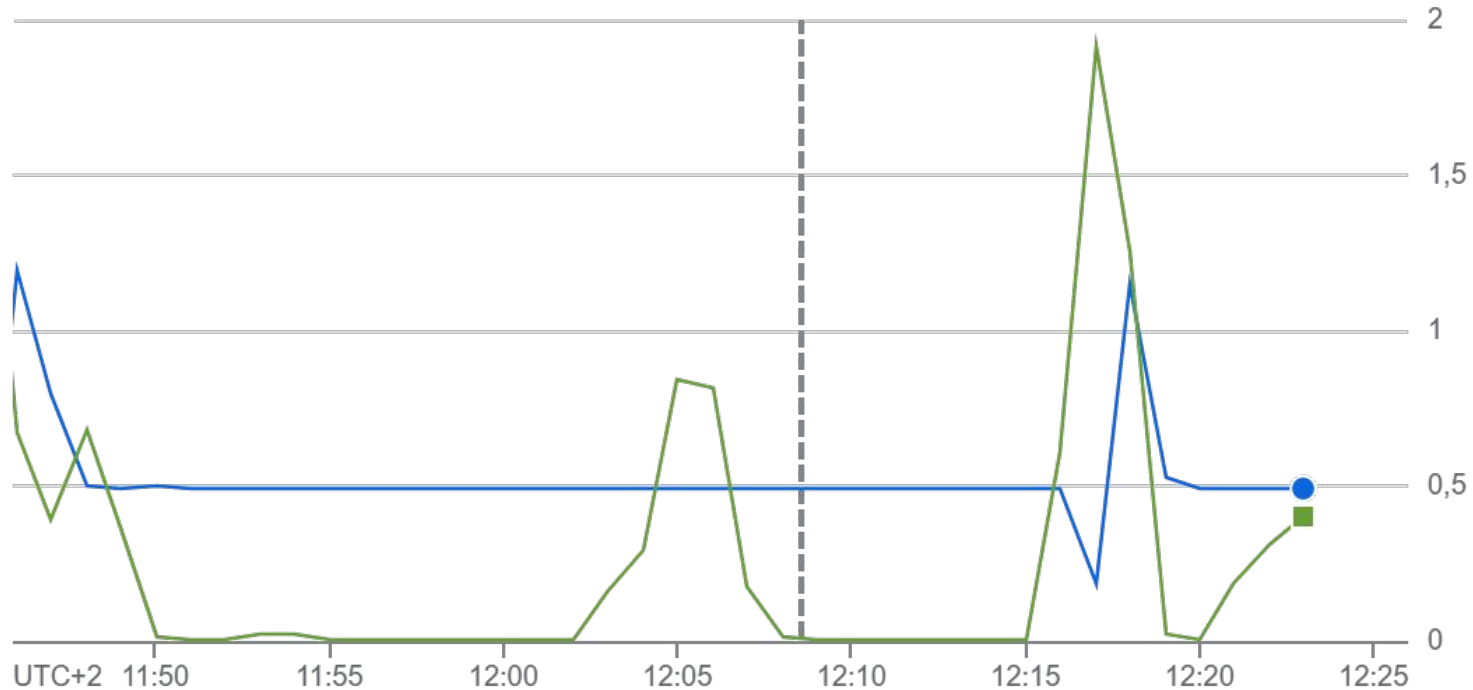
20 users, 2 ramp up - small edge - standard cloud

VM Instance - CPU utilization [MEAN]



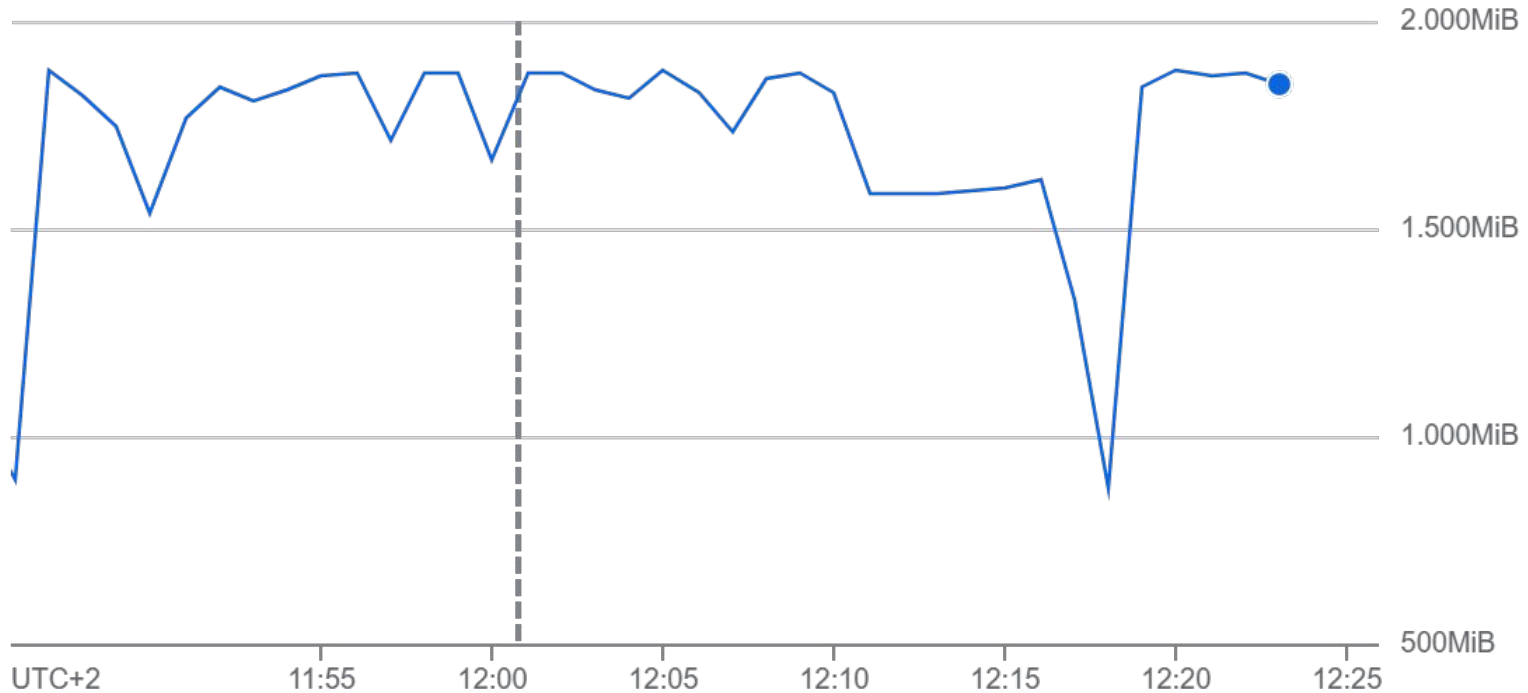
20 users, 2 spawn rate - small edge - standard cloud

VM Instance - CPU usage [RATE]



20 users, 2 spawn rate - small edge - standard cloud

VM Memory Used [MEAN]



20 users, 2 spawn rate - small edge - standard cloud



Host

Status

running

Users

20

RPS

1.6

Failures

59%

Edit

Stop

Reset

Statistics

Charts

Failures

Exceptions

Current Ratio

Download Data

Logs

Locust Cloud

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
workflow	full_workflow	17	10	1477000	1480000	1480000	1476154.12	1471563	1480280	5903.94	1.6	1
	Aggregated	17	10	1477000	1480000	1480000	1476154.12	1471563	1480280	5903.94	1.6	1

About

Possible Improvements

- allow for automatic scaling of instances to split the workload
- use hardware optimized for image classification
- optimize the flower finder model

Thank you for your
attention!